Earned Autonomy: A Reversibility and Evidence Gate for Automated Infrastructure Remediation

Aman Das
Earned Autonomy Project
Remote, India
Supervisor: Kother Badushah

***Abstract*—Automated remediation can reduce the time required to recover from infrastructure failures, but allowing an automated system to act without sufficient evidence can introduce additional failures or increase the impact of an incorrect decision. This paper presents EarnedAutonomy, a trust gate for self-healing infrastructure that determines whether a candidate remediation action should be executed automatically, proposed for human approval, or escalated to a human operator. The gate evaluates three factors: action reversibility, blast radius, and accumulated evidence of previous outcomes. Evidence is represented using a Beta posterior distribution and evaluated using a conservative lower posterior bound rather than the posterior mean. Evidence is maintained separately for each action type and machine class so that successful outcomes in one context do not automatically transfer to another. The project uses the public Blue Gene/L system-log dataset to inform the simulated incident distribution and evaluates the gate using controlled simulation. The implementation includes configurable action and threshold parameters, Bayesian evidence tracking, outcome feedback, automated safety tests, baseline benchmarking, and threshold sensitivity experiments. The experiments demonstrate that the gate can permit substantially less autonomous execution than an always-act policy while reducing unsafe automatic executions, although this conservative behavior also increases human involvement and mean time to recovery. The results therefore illustrate the intended safety-oriented trade-off rather than claiming that EarnedAutonomy dominates all baseline metrics.**

***Index Terms*—Automated remediation, self-healing infrastructure, Bayesian evidence, trust gate, infrastructure reliability, autonomous systems**

I. Introduction

Modern infrastructure systems increasingly use automation to detect failures and initiate remediation actions. Automation can reduce recovery time and decrease the operational workload placed on human engineers. However, an automation system that executes every proposed action without considering context can turn an uncertain diagnosis into an unsafe infrastructure change.

The central problem addressed by this project is therefore not failure detection itself, but the decision of whether an already proposed remediation action has earned sufficient trust to be executed automatically.

The project prioritizes limiting unsafe automatic executions rather than maximizing the number of actions executed automatically. EarnedAutonomy introduces an intermediate trust layer between a fault detector and an action executor. The gate can return one of three decisions:

- **AUTO**: the candidate action may be executed automatically.
- **PROPOSE**: the candidate action may be proposed to a human operator for approval.
- **ESCALATE**: the incident should remain under human control.

An action can initially require human involvement and later become eligible for automatic execution as successful evidence accumulates. Conversely, unsuccessful outcomes can reduce the accumulated evidence and prevent continued automatic execution.

The design intentionally uses three factors. Reversibility represents how easily an action can be undone. Blast radius represents the potential scope of impact if an action is incorrect. Accumulated evidence represents historical observations of whether a particular action has successfully resolved incidents in a particular machine-class context.

The evaluation is designed around a safety-first objective. In particular, unsafe automatic executions are treated as a primary negative outcome rather than optimizing solely for the number of actions executed automatically. This reflects the operational principle that a self-healing system should earn autonomy gradually rather than assume that automation is inherently desirable.

II. Problem Statement

A conventional automated remediation workflow can be represented as:

$$\text{Failure Detection} \rightarrow \text{Candidate Action} \rightarrow \text{Action Execution}. \tag{1}$$

EarnedAutonomy inserts a trust decision between the candidate action and its execution:

$$\text{Failure Detection} \rightarrow \text{Candidate Action} \rightarrow \text{EarnedAutonomy Gate} \rightarrow \text{Deci} \tag{2}$$

The decision is:

$$\text{Decision} \in \{\text{AUTO}, \text{PROPOSE}, \text{ESCALATE}\}. \tag{3}$$

The missing component in a conventional workflow is therefore a mechanism that determines whether an action

TABLE I
BGL DATASET SUMMARY

Property	Value
Total records	4,747,963
Normal records	4,399,503
Alert records	348,460
Alert percentage	7.34%
Distinct labels	42

has accumulated sufficient evidence and has sufficiently low operational risk to justify autonomous execution.

An action that is appropriate in one context may not be appropriate in another. For example, restarting a stateless service may be relatively reversible, while draining a compute node may affect a larger amount of active work. Similarly, an action that has repeatedly succeeded on one machine class should not automatically inherit the same confidence when applied to a different machine class.

EarnedAutonomy addresses this problem by evaluating a candidate action using reversibility, blast radius, and accumulated evidence before determining the level of autonomy permitted.

III. PROJECT SCOPE

The project is deliberately limited to the decision layer governing remediation actions.

EarnedAutonomy does not:

- detect infrastructure failures;
- execute remediation actions on real infrastructure;
- build an infrastructure monitoring dashboard;
- use machine learning to detect failures;
- replace an existing remediation executor;
- introduce additional trust factors beyond the specified design.

The intended architecture is:

$$\begin{aligned} \text{Fault Detector} &\rightarrow \text{Candidate Action} \\ &\rightarrow \text{EarnedAutonomy Gate} \\ &\rightarrow \text{Decision}. \end{aligned} \tag{4}$$

The trust gate therefore acts as a control point between detecting a problem and permitting an automated response.

IV. DATASET ANALYSIS

A. Public Log Dataset

The project uses the Blue Gene/L (BGL) system log dataset as a public source for deriving the distribution of incident types used by the simulator [2]. The dataset is used to inform the shape of the simulated incident stream rather than to train a failure-detection model.

The analyzed dataset contains 4,747,963 log records. Of these, 4,399,503 are normal records and 348,460 are alert records, corresponding to an alert rate of approximately 7.34%. The dataset contains 42 distinct alert labels.

The most frequent observed labels include KERNDTLB, KERNSTOR, APPSEV, KERNMNTF, and KERNTERM. These observations were used to inform the simplified incident taxonomy and incident-type weights used by the simulator.

The dataset is therefore used as an empirical source for simulator design rather than as a source of remediation ground truth. In particular, the BGL data does not directly provide a mapping from each incident label to a particular remediation action.

B. Incident Taxonomy

The simulator represents five broad incident types:

- kernel error;
- storage error;
- filesystem failure;
- communication failure;
- application error.

The mapping is intentionally broader than the original log labels. The purpose is to create a small and interpretable incident space for evaluating the trust gate rather than reproducing the complete BGL logging system.

V. INCIDENT AND ACTION MODELING

A. Incident Model

Each simulated incident contains:

- an incident identifier;
- an incident type;
- a machine class;
- a timestamp.

The simulator currently uses three machine classes:

- `compute_node`;
- `storage_node`;
- `scheduler`.

The initial simulator verification generates 1,000 incidents.

B. Action Types

The project defines six remediation action types. The action descriptions follow the project specification, while the reversibility and blast-radius values are the configuration values used by the implementation.

TABLE II
REMEDIATION ACTIONS AND INITIAL FACTORS

Action	Reversibility	Blast Radius
Restart stateless service	0.9	0.1
Requeue failed job	0.9	0.1
Reset device driver	0.6	0.3
Drain node and migrate work	0.5	0.7
Reclaim disk space	0.3	0.5
Escalate to human	1.0	0.0

The values are normalized to the range $[0, 1]$. Reversibility is higher when an action can be easily undone. Blast radius is higher when an incorrect action can affect a larger portion of the system.

VI. Simulator

The simulator generates incidents according to the simplified incident taxonomy. The current incident-type probabilities are:

TABLE III
Simulator Incident-Type Weights

Incident Type	Weight
Kernel error	0.5235
Storage error	0.1831
Filesystem failure	0.0930
Communication failure	0.0177
Application error	0.1827

These weights are modeling assumptions informed by the observed BGL label distribution. They are not intended to represent an exact mapping from BGL labels to infrastructure failure causes.

Figure 1 shows the distribution produced by the simulator for the initial 1,000-incident run.

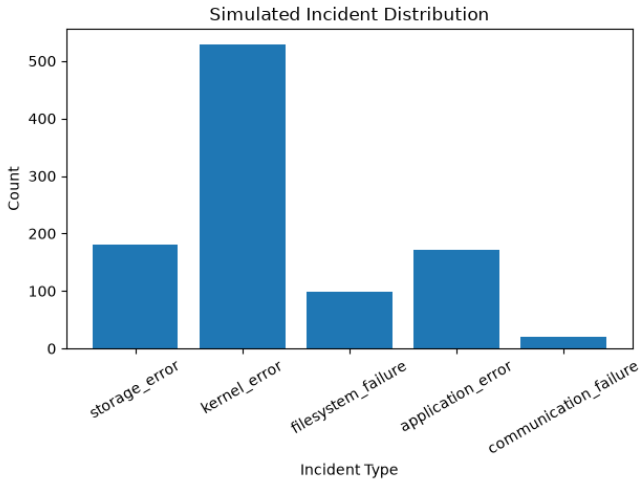


Fig. 1. Simulated incident distribution for the initial 1,000 generated incidents.

The simulator also models remediation outcomes so that the evidence component can learn from successful and unsuccessful actions. Outcome feedback is evaluated separately from the incident-distribution model described above.

VII. Earned Autonomy Trust Gate

A. Three Factors

The trust gate combines three factors.

1) Reversibility: Reversibility represents how easily a remediation action can be undone. Actions with higher reversibility are treated as safer candidates for automation.

2) Blast Radius: Blast radius represents the potential scope of impact if an action is incorrect. A larger blast radius increases the risk associated with automatic execution.

3) Evidence: Evidence represents the accumulated history of whether an action has successfully resolved previous incidents.

The evidence is maintained separately for each combination of:

$$(\text{action type}, \text{machine class}). \tag{5}$$

This context separation prevents successful outcomes from one machine class from automatically becoming evidence for another machine class.

B. Bayesian Evidence Model

Evidence is represented using a Beta posterior distribution [5]. Each action–machine-class combination starts with a uniform prior:

$$p \sim \text{Beta}(1, 1). \tag{6}$$

A successful remediation increments the α parameter:

$$\alpha \leftarrow \alpha + 1. \tag{7}$$

An unsuccessful remediation increments the β parameter:

$$\beta \leftarrow \beta + 1. \tag{8}$$

The posterior mean is:

$$E[p] = \frac{\alpha}{\alpha + \beta}. \tag{9}$$

However, the trust gate does not use the posterior mean as its evidence score. Instead, it uses a conservative lower posterior percentile. The current implementation uses the 5th percentile of the Beta posterior:

$$E_{LB} = F^{-1}_{\text{Beta}(\alpha,\beta)}(0.05). \tag{10}$$

This conservative estimate makes the gate less willing to grant autonomy when the evidence is limited or uncertain. The implementation uses the SciPy Beta distribution inverse cumulative distribution function to calculate this value directly.

C. Trust and Risk

The trust score combines reversibility with the conservative evidence estimate:

$$\text{trust} = \text{reversibility} \times E_{LB}. \tag{11}$$

The risk score is the normalized blast radius:

$$\text{risk} = \text{blast radius}. \tag{12}$$

The current implementation uses the following configurable thresholds:

$$\text{AUTO_THRESHOLD} = 0.65 \tag{13}$$

$$\text{PROPOSE_THRESHOLD} = 0.35 \tag{14}$$

$$\text{RISK_CEILING} = 0.60. \tag{15}$$

These values are engineering configuration choices for the current implementation. Their effect is examined later through threshold sensitivity experiments.

D. Decision Logic

The current gate applies the following decision rule:

$$\begin{cases} \text{AUTO}, & h \wedge (\text{trust} > T_A) \wedge (\text{risk} < R_C) \\ \text{PROPOSE}, & \text{trust} > T_P \\ \text{ESCALATE}, & \text{otherwise.} \end{cases} \tag{16}$$

Here, h indicates whether the action–machine-class combination has previously observed an outcome.

The history requirement ensures that an action with no observed outcomes cannot be automatically executed. The risk ceiling provides an additional constraint on automatic execution, so an action cannot become AUTO solely by accumulating successful outcomes when its blast radius is too high.

This produces three possible autonomy levels: AUTO permits automatic execution, PROPOSE requests human approval, and ESCALATE requires human handling.

VIII. Implementation

The implementation is organized into small components:

- `src/gate.py` implements the trust gate and decision logic.
- `src/evidence.py` implements Beta evidence tracking and lower-bound calculation.
- `src/simulator.py` generates simulated incidents and outcome feedback.
- `src/baselines.py` implements the benchmark baseline policies.
- `config/actions.yaml` stores action reversibility and blast-radius values.
- `config/thresholds.yaml` stores the decision thresholds.
- `benchmark.py` executes the baseline and EarnedAutonomy benchmark.
- `threshold_sweep.py` evaluates sensitivity to decision thresholds.
- `run_learning_experiment.py` evaluates evidence accumulation and generates the autonomy progression plot.
- `verify_gate.py` performs end-to-end gate verification.

The implementation uses Python together with NumPy, pandas, SciPy, PyYAML, matplotlib, and pytest.

The configuration values are kept outside the core decision code so that action factors and thresholds can be changed without modifying the gate logic.

IX. Initial Gate Verification

The initial trust-gate implementation was verified using automated tests covering the required safety properties of the gate.

The tests verify that:

- an action with no history never returns AUTO;
- a high-blast-radius action does not return AUTO;
- moderate evidence can produce a PROPOSE decision;
- sufficiently strong evidence for a low-risk action can produce AUTO;
- insufficient evidence produces ESCALATE.

The final repository test suite contains ten tests covering the gate and benchmark behavior. The latest verification run reported:

$$10 \text{ tests passed.} \tag{17}$$

The implementation was also run end-to-end on 1,000 simulated incidents. Because this initial verification run does not provide remediation outcomes, no action–machine-class combination has accumulated evidence. As a result, all 1,000 decisions were ESCALATE.

TABLE IV
Initial Decision Counts for 1,000 Simulated Incidents

Decision	Count
AUTO	0
PROPOSE	0
ESCALATE	1,000

X. Outcome Modeling and Evidence Feedback

The simulator was extended to provide observable remediation outcomes. This feedback is required because the evidence component of the trust gate must update its estimate of remediation reliability from observed results.

Three outcome types are modeled:

- **RESOLVED**: the remediation action successfully resolves the simulated incident;
- **RECURRED**: the action appears to work initially, but the underlying problem returns;
- **HARMFUL**: the remediation action produces a harmful outcome for the simulated incident.

For the controlled evidence-accumulation experiment, outcome probabilities are conditioned on candidate action type rather than estimated from the BGL dataset.

For `restart_service`, the simulator uses:

$$P(\text{RESOLVED}) = 0.80, \quad P(\text{RECURRED}) = 0.15, \quad P(\text{HARMFUL}) \tag{18}$$

For `drain_node`, it uses:

$$P(\text{RESOLVED}) = 0.50, \quad P(\text{RECURRED}) = 0.25, \quad P(\text{HARMFUL}) \tag{19}$$

These probabilities are fixed simulation assumptions used to create controlled differences in evidence accumulation. They are not probabilities estimated from the BGL dataset.

In the evidence-accumulation experiment, an outcome is generated after each gate decision, including when the gate returns `ESCALATE`. This represents human-controlled or human-approved handling of the candidate action whose observed result can still contribute evidence for future decisions. No real infrastructure action is executed.

The outcome is fed back into the evidence store using the same action-type and machine-class context used by the trust gate. A `RESOLVED` outcome is treated as a successful observation and increments the Beta posterior parameter α. Both `RECURRED` and `HARMFUL` outcomes are treated as failed observations and increment β:

$$\text{RESOLVED} \Rightarrow \alpha \leftarrow \alpha + 1 \tag{20}$$

$$\text{RECURRED or HARMFUL} \Rightarrow \beta \leftarrow \beta + 1. \tag{21}$$

This creates the feedback loop required for earned autonomy. Observed remediation outcomes modify the evidence associated with an action–machine-class combination, and the updated evidence affects subsequent gate decisions.

XI. Evidence Accumulation Evaluation

A controlled learning experiment was conducted using deterministic random seed 42 and 100 simulated incidents for each of two action contexts. Both experiments used the `compute_node` machine class.

The first experiment evaluated `restart_service`. The action has reversibility 0.9 and blast radius 0.1. Its action-conditioned outcome model uses probabilities of 0.80 for `RESOLVED`, 0.15 for `RECURRED`, and 0.05 for `HARMFUL`.

The second experiment evaluated `drain_node`. The action has reversibility 0.5 and blast radius 0.7. Since the configured risk ceiling is 0.60, the action cannot satisfy the automatic-execution risk condition regardless of how much positive evidence accumulates.

TABLE V
Evidence Accumulation Experiment Results

Action	ESCALATE	PROPOSE	AUTO
Restart service	3	50	47
Drain node	100	0	0

For `restart_service`, the first AUTO decision occurred at simulated incident 16. The running proportion of AUTO decisions increased as additional observations accumulated, reaching 47% after 100 simulated incidents. The final count was 47 AUTO, 50 PROPOSE, and 3 ESCALATE decisions.

For `drain_node`, no AUTO decision occurred during the 100 simulated incidents. The gate produced 100 ESCALATE decisions. This behavior is consistent with both the lower reversibility and, more importantly, the configured blast-radius ceiling.

Figure 2 shows the running proportion of AUTO decisions across the two experiments. The plotted value at each incident is the proportion of decisions classified as AUTO up to that point; it is not a prediction of future automation rate.

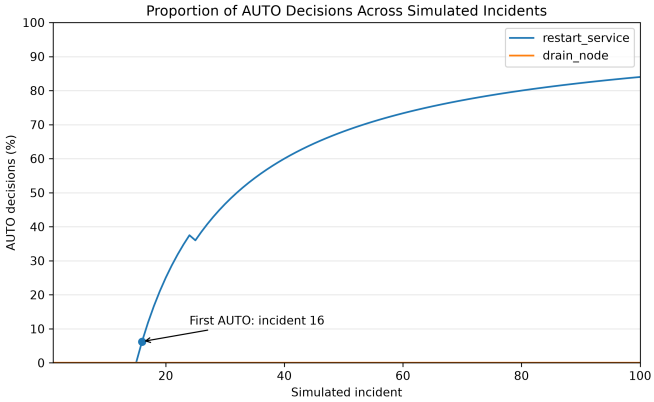


Fig. 2. Running proportion of AUTO decisions during the evidence-accumulation experiment. Restart service reaches AUTO after sufficient evidence, while drain node remains at zero AUTO decisions.

The experiment demonstrates the intended feedback behavior in the controlled simulator: a low-risk action with accumulating evidence can progress toward automatic execution, while a high-blast-radius action remains excluded from AUTO.

XII. Experimental Methodology

The benchmark evaluates three policies under the same simulated incident stream:

- **Always Ask**: every candidate remediation requires human intervention;
- **Always Act**: every candidate remediation is executed automatically;
- **EarnedAutonomy**: every candidate remediation is evaluated by the trust gate.

The comparison uses four primary metrics:

1) automatic resolution rate;
2) mean time to recovery;
3) human interruptions;
4) unsafe automatic executions.

The benchmark uses an explicit simulated incident-to-candidate-action mapping. This mapping is a simulation assumption because the BGL dataset provides incident-log information rather than remediation ground truth.

The benchmark uses deterministic random seeds so that the same simulated conditions can be reproduced.

The evaluation is safety-oriented. In particular, unsafe automatic executions are treated as a primary negative outcome rather than optimizing solely for the number of automatically executed actions.

XIII. Baseline Benchmark

The first benchmark run used random seed 42. Table VI reports the resulting comparison.

TABLE VI
BENCHMARK RESULTS FOR SEED 42

Policy	Automatic Resolution	MTTR (s)	Human Interruptions	Unsafe Automatic
Always Ask	0.000	291.685	1000	0
Always Act	0.528	171.685	472	472
EarnedAutonomy	0.000	443.905	1000	2

For seed 42, Always Act achieved an automatic resolution rate of 0.528 and a mean time to recovery of 171.685 seconds, but produced 472 unsafe automatic executions.

Always Ask produced no automatic resolutions and required human intervention for all 1,000 incidents.

EarnedAutonomy permitted only two autonomous executions in this run. Both were unsafe, resulting in two unsafe automatic executions. The gate therefore substantially restricted autonomous execution compared with Always Act, although it did not achieve an automatic resolution in this particular seed.

A second deterministic benchmark run used seed 43.

TABLE VII
BENCHMARK RESULTS FOR SEED 43

Policy	Automatic Resolution	MTTR (s)	Human Interruptions	Unsafe Automatic
Always Ask	0.000	288.300	1000	0
Always Act	0.538	168.300	462	462
EarnedAutonomy	0.011	442.320	989	0

For seed 43, EarnedAutonomy permitted 11 autonomous executions, corresponding to an automation rate of 1.1%. It achieved an automatic resolution rate of 0.011 and recorded zero unsafe automatic executions.

Always Act again achieved the highest automatic resolution rate but also produced 462 unsafe automatic executions.

Across the two seeds, the qualitative behavior is consistent: EarnedAutonomy permits substantially less autonomous execution than Always Act, while the absolute number of unsafe autonomous executions is much lower. The trade-off is increased human involvement and higher mean time to recovery.

XIV. Threshold Sensitivity Analysis

The decision thresholds are configurable and therefore require sensitivity analysis. The threshold experiment evaluates combinations of AUTO and PROPOSE thresholds while maintaining the configured risk ceiling.

The AUTO threshold values evaluated were:

$$\{0.50, 0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95\}. \tag{22}$$

The PROPOSE threshold values evaluated were:

$$\{0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50\}. \tag{23}$$

Configurations in which the PROPOSE threshold was greater than or equal to the AUTO threshold were excluded.

The sweep therefore evaluates only logically ordered configurations in which the AUTO threshold remains above the PROPOSE threshold.

For seed 42 and a PROPOSE threshold of 0.35, the observed relationship between AUTO threshold, automation, and unsafe automatic executions is shown in Table VIII.

TABLE VIII
SELECTED THRESHOLD-SWEEP RESULTS FOR SEED 42 AT PROPOSE = 0.35

AUTO Threshold	Automation Rate	Unsafe Automatic
0.50	0.037	9
0.55	0.025	7
0.60	0.011	4
0.65	0.002	2
0.70	0.000	0
0.75	0.000	0
0.80	0.000	0
0.85	0.000	0
0.90	0.000	0
0.95	0.000	0

The threshold sweep demonstrates the expected safety–automation trade-off. Lowering the AUTO threshold increases autonomous execution but also increases unsafe automatic executions in the simulated environment. Raising the AUTO threshold eventually eliminates autonomous execution entirely.

The configured AUTO threshold of 0.65 therefore represents a conservative operating point in the current simulator rather than a universally optimal threshold.

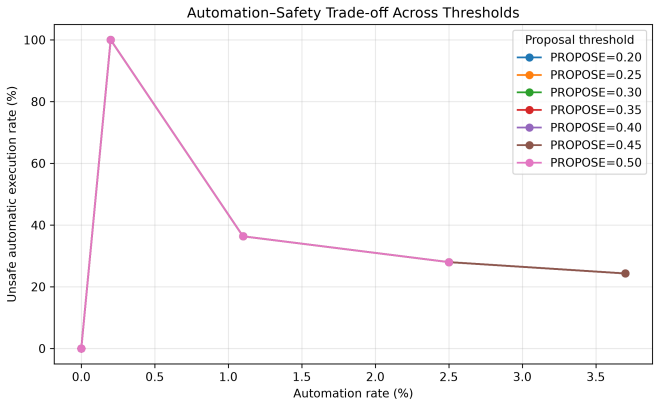


Fig. 3. Threshold sensitivity for seed 42 showing the relationship between autonomous execution and unsafe automatic executions.

The same threshold sweep was also evaluated using seed 43.

The threshold results reinforce the safety-oriented objective of the project: increasing automation is not treated as an unconditional improvement. A threshold configuration that increases autonomous execution while also increasing unsafe automatic executions may be undesirable for a safety-oriented remediation gate.

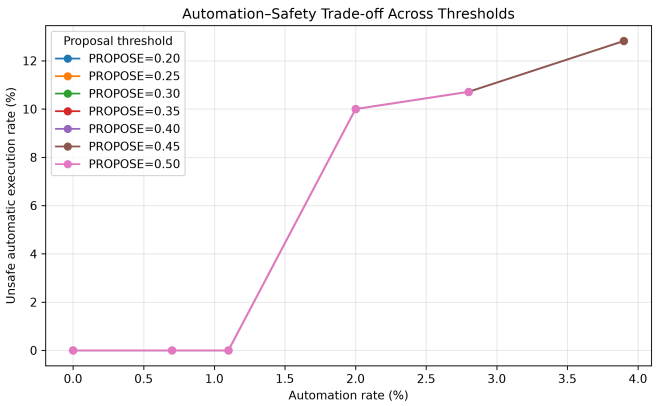


Fig. 4. Threshold sensitivity for seed 43 under a second deterministic simulated incident sequence.

XV. Discussion

The experimental results show that EarnedAutonomy behaves differently from both simple baselines.

Always Ask provides the strongest conservative policy in terms of unsafe automatic executions because it performs no autonomous remediation. Its cost is complete dependence on human intervention and the absence of automatic resolution.

Always Act provides the highest level of automation and the lowest simulated mean time to recovery among the evaluated policies. However, this policy also produces a large number of unsafe automatic executions. In both benchmark seeds, the unsafe automatic execution count is hundreds of incidents.

EarnedAutonomy occupies an intermediate position. It does not attempt to maximize autonomous execution. Instead, it restricts autonomous execution until the combination of reversibility, evidence, and risk satisfies the gate conditions.

For seed 42, this conservative behavior resulted in only two autonomous executions, both of which were unsafe. For seed 43, the gate produced 11 autonomous executions and none were unsafe.

These two runs illustrate an important property of the evaluation: the gate can substantially reduce exposure to unsafe automatic actions, but low autonomous execution can also reduce the number of incidents resolved automatically.

The results therefore should not be interpreted as evidence that EarnedAutonomy universally outperforms the baselines. Rather, they demonstrate the intended safety-oriented trade-off and provide a reproducible framework for examining how the trade-off changes with threshold selection and simulated outcomes.

XVI. When the Gate Does Not Dominate

The benchmark identifies at least two conditions in which EarnedAutonomy does not dominate the simpler baselines.

First, when the primary objective is minimum mean time to recovery and unsafe automation is ignored, Always Act performs better in the current simulation. For seed 42, Always Act achieves 171.685 seconds compared with 443.905 seconds for EarnedAutonomy. For seed 43, Always Act achieves 168.300 seconds compared with 442.320 seconds for EarnedAutonomy.

Second, when the primary objective is maximum automatic resolution rate, Always Act also performs better. Its automatic resolution rate is 0.528 for seed 42 and 0.538 for seed 43, while EarnedAutonomy achieves 0.000 and 0.011, respectively.

These results are important because the purpose of the gate is not to dominate every metric. Its design objective is to control the safety risk associated with autonomous remediation.

XVII. Limitations and Threats to Validity

The current implementation has several limitations.

First, the project uses a simulator rather than a production infrastructure environment. Simulated outcomes cannot fully reproduce the complexity of real remediation scenarios.

Second, the machine classes are simplified into three broad categories: compute nodes, storage nodes, and schedulers. Real infrastructure may require a more detailed contextual model.

Third, the reversibility and blast-radius values are configuration assumptions rather than measurements from production observations. These values may therefore differ from those of real remediation systems.

Fourth, the BGL dataset is used to inform the shape of the simulated incident distribution rather than to represent a direct production environment. The simplified incident taxonomy and weighting therefore do not reproduce the complete characteristics of the original dataset.

Fifth, the simulator does not provide production incident-to-remediation ground truth. The benchmark therefore uses an explicit simulated mapping between incidents and candidate actions. This is a necessary modeling assumption for controlled evaluation and is not claimed to originate from the BGL dataset.

Sixth, the action-conditioned outcome probabilities are simulation assumptions. They are not learned from production telemetry.

Seventh, the evidence-accumulation experiment allows outcomes generated after ESCALATE decisions to contribute evidence. This represents human-controlled handling of the candidate action in the simulation and is not equivalent to autonomous execution.

Finally, the benchmark results are based on deterministic simulated seeds and should not be interpreted as evidence of production performance. The current experiments demonstrate the behavior and trade-offs of the decision mechanism rather than validating its effectiveness on real infrastructure.

XVIII. Reproducibility

The project is implemented as a Python repository with configuration, source code, tests, benchmark scripts, experiment outputs, and documentation.

The core gate is implemented in:

- `src/gate.py`;
- `src/evidence.py`;

- `src/simulator.py`.

The benchmark is implemented in `benchmark.py`. Baseline policies are implemented in `src/baselines.py`. Threshold sensitivity experiments are implemented in `threshold_sweep.py`.

The evidence-accumulation experiment is implemented in `run_learning_experiment.py`. The experiment uses random seed 42 and evaluates 100 simulated incidents for each tested action.

The initial gate verification can be reproduced using the repository's Python test suite and `verify_gate.py`.

The benchmark can be reproduced using:

```
python benchmark.py
```

The threshold sweep can be reproduced using:

```
python threshold_sweep.py --seed 42
python threshold_sweep.py --seed 43
```

The evidence-accumulation experiment can be reproduced using:

```
python run_learning_experiment.py
```

The test suite can be executed using:

```
python -m pytest -q
```

The final verification reported ten passing tests.

The public BGL dataset is not stored in the repository. The benchmark and threshold-sweep outputs are stored under the repository's `results/` directory.

XIX. Conclusion and Future Work

EarnedAutonomy introduces a trust decision layer for automated infrastructure remediation. The implementation combines reversibility, blast radius, and context-specific Bayesian evidence to determine whether a candidate action should be automatically executed, proposed for human approval, or escalated to a human operator.

The initial gate verification establishes two important safety properties. First, an action with no observed history cannot become AUTO. Second, a high-blast-radius action remains excluded from AUTO by the configured risk ceiling.

The evidence-accumulation experiment demonstrates the intended earned-autonomy behavior. The low-risk `restart_service` action progresses from human-controlled decisions toward AUTO as evidence accumulates, while the high-blast-radius `drain_node` action remains excluded from AUTO.

The benchmark experiments demonstrate the trade-off between automation and safety. Always Act achieves higher automatic resolution and lower simulated recovery time, but it also produces hundreds of unsafe automatic executions. EarnedAutonomy substantially restricts autonomous execution and consequently reduces unsafe exposure, but this comes with increased human involvement and higher mean time to recovery.

The threshold sensitivity experiments further show that increasing the AUTO threshold reduces autonomous execution and eventually eliminates unsafe automatic executions in the simulated environment. This supports the design principle that autonomy should be earned through evidence and constrained by action risk rather than maximized by default.

The results are limited to the controlled simulator. The outcome probabilities, incident-to-action mapping, reversibility values, and blast-radius values are simulation assumptions rather than measurements from production infrastructure. The results therefore demonstrate the behavior of the decision mechanism rather than production-level remediation performance.

Future work can extend the evaluation to richer incident-to-action relationships, calibrated action risk estimates, larger machine-class taxonomies, and eventually controlled real-world infrastructure experiments. Any future deployment should preserve the primary safety objective of minimizing unsafe automatic executions.

Acknowledgment

The author thanks Kother Badushah for supervision and guidance throughout the project.

References

[1] A. Oliner and J. Stearley, "What Supercomputers Say: A Study of Five System Logs," in *Proc. IEEE International Conference on Dependable Systems and Networks*, 2007.

[2] A. Oliner, "Blue Gene/L Supercomputer Logs," LogHub / Zenodo, dataset record 8196385.

[3] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[4] M. C. Huebscher and J. A. McCann, "A Survey of Autonomic Computing—Degrees, Models, and Applications," *ACM Computing Surveys*, vol. 40, no. 3, 2008.

[5] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin, *Bayesian Data Analysis*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2013.